

Persona-Based Multi-Agent Architecture

Using Google ADK Framework + PostgreSQL

Overview

This document describes a scalable, reusable multi-agent system architecture where **tools**, **skills**, and **agent personas** are stored in PostgreSQL and dynamically loaded at runtime via the Google ADK framework. The design allows hundreds of use cases to share common logic without duplicating code.

Alignment with Agent Skills Open Standard (agentskills.io)

This architecture is designed to be **fully compliant** with the [Agent Skills open standard](#), released by Anthropic in December 2025 and adopted across Claude, OpenAI Codex, Gemini CLI, GitHub Copilot, VS Code, and others.

What the Standard Requires

Each skill must be a **directory** containing at minimum a `SKILL.md` file with YAML frontmatter:

Field	Required	Constraints
<code>name</code>	Yes	Max 64 chars, lowercase, hyphens only
<code>description</code>	Yes	Max 1024 chars, describes what the skill does and when to use it
<code>license</code>	No	License name or reference to bundled file
<code>compatibility</code>	No	Max 500 chars, environment requirements
<code>metadata</code>	No	Arbitrary key-value map
<code>allowed-tools</code>	No	Space-separated pre-approved tools (experimental)

How Your Skill Registry Maps to the Standard

Your Registry Field	Agent Skills Standard Field	Notes
skill_name	name	Must be lowercase-hyphenated e.g. fetch-source-data
description	description	Must include WHAT it does AND WHEN to trigger it
instruction	SKILL.md body content	Step-by-step instructions for the agent
input_schema	assets/input-schema.json	Move to assets/ directory
output_schema	assets/output-schema.json	Move to assets/ directory
tool_id	allowed-tools	Reference the bound tool name here
(missing)	metadata.version	Add versioning to your skill records
(missing)	metadata.author	Add ownership tracking

Compliant Skill Directory Structure

Each skill in your registry should also exist as a directory on disk so it is portable across any Agent Skills-compatible client:

```
skills/
├─ fetch-source-data/
│   ├── SKILL.md           # Required: frontmatter + instructions
│   ├── assets/
│   │   ├── input-schema.json
│   │   └─ output-schema.json
│   └─ scripts/
│       └─ fetch.py        # Optional helper
├─ compare-two-sources/
│   ├── SKILL.md
│   └─ assets/
│       └─ input-schema.json
└─ verdict-summary/
```

Compliant SKILL.md Example

```
---
name: fetch-source-data
description: >
  Fetches structured data from a configured database or API tool.
  Use when an agent needs to retrieve raw data from a source system
  (e.g. PostgreSQL query, REST API GET) before comparison or analysis.
license: Proprietary
compatibility: Requires a bound tool from the tools registry (postgres_query or r
metadata:
  author: qa-platform-team
  version: "1.0"
allowed-tools: postgres_query rest_api_get
---
```

Instructions

You are a precise data retrieval agent. Fetch data as instructed and return it wi

Steps

1. Receive the tool configuration and query/URL from runtime parameters
2. Execute the tool with the provided parameters
3. Return the raw result set as a JSON array
4. Do not transform, filter, or interpret the data

Input

See [input schema](assets/input-schema.json) for required parameters.

Edge Cases

- If query returns zero rows, return an empty array `[]`
- If the tool call fails, return `{"error": "...", "tool": "..."}`

Gaps and Required Corrections

1. Skill names must use lowercase hyphens, not underscores. Change

fetch_source_a_data → fetch-source-data .

2. Skills must also exist as file-system directories. Your PostgreSQL registry handles runtime, but each skill should also be a SKILL.md directory for portability across other

Agent Skills clients (VS Code, Codex, etc.).

3. Add metadata , license , compatibility , and allowed_tools fields to your skills table:

```
ALTER TABLE skills
  ADD COLUMN metadata      JSONB DEFAULT '{}',
  ADD COLUMN license       VARCHAR(200),
  ADD COLUMN compatibility VARCHAR(500),
  ADD COLUMN allowed_tools TEXT;  -- space-separated tool names
```

4. Descriptions must include both WHAT and WHEN. Current descriptions only describe what the skill does. Add trigger conditions like “Use when...” to every description.

5. Validate skills using the official CLI:

```
npm install -g skills-ref
skills-ref validate ./skills/fetch-source-data
```

Revised Standard-Aligned Skills Table

```
CREATE TABLE skills (
  skill_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  skill_name        VARCHAR(100) UNIQUE NOT NULL,  -- e.g. "fetch-source-data"
  tool_id           UUID REFERENCES tools(tool_id),
  description        TEXT NOT NULL,                -- max 1024 chars, includes WH
  instruction        TEXT NOT NULL,                -- SKILL.md body content
  input_schema       JSONB NOT NULL,
  output_schema      JSONB,
  allowed_tools      TEXT,                          -- space-separated (allowed-to
  metadata           JSONB DEFAULT '{}',           -- version, author, etc.
  license            VARCHAR(200),
  compatibility       VARCHAR(500),
  created_at         TIMESTAMP DEFAULT NOW()
);
```

Core Concepts

The Three Registries

Registry	What It Stores	Key Property
----------	----------------	--------------

Tools Registry	Reusable integrations (DB queries, API calls, file readers)	Stateless, parameterizable
Skills Registry	Templates that bind a tool + expected parameters	No hard-coded values
Agent / Persona Registry	Role definitions with assigned skill sets	Composable, reusable across use cases

Registry Definitions

1. Tools Registry

A tool is a reusable integration. It knows *how* to connect to something but does not know *what* to fetch.

```
CREATE TABLE tools (
  tool_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tool_name        VARCHAR(100) UNIQUE NOT NULL,      -- e.g. "postgres_query", "res
  tool_type        VARCHAR(50)  NOT NULL,             -- "database", "api", "file",
  description      TEXT,
  config_schema    JSONB NOT NULL,                    -- JSON Schema for required co
  created_at       TIMESTAMP DEFAULT NOW()
);
```

Example Tool Records:

```
{
  "tool_name": "postgres_query",
  "tool_type": "database",
  "config_schema": {
    "type": "object",
    "required": ["connection_string", "query"],
    "properties": {
      "connection_string": { "type": "string" },
      "query": { "type": "string" },
      "params": { "type": "array" }
    }
  }
}
```

```
{
```

```

"tool_name": "rest_api_get",
"tool_type": "api",
"config_schema": {
  "required": ["url", "headers"],
  "properties": {
    "url": { "type": "string" },
    "headers": { "type": "object" },
    "timeout_seconds": { "type": "integer", "default": 30 }
  }
}
}

```

2. Skills Registry

A skill is a **template** — it declares which tool it needs and what parameter *slots* it expects, but never hard-codes values.

```

CREATE TABLE skills (
  skill_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  skill_name        VARCHAR(100) UNIQUE NOT NULL,    -- e.g. "fetch_source_a_data"
  tool_id           UUID REFERENCES tools(tool_id),
  instruction        TEXT NOT NULL,                  -- Agent instruction template
  input_schema       JSONB NOT NULL,                  -- Parameters this skill expects
  output_schema      JSONB,                          -- Expected output shape
  description        TEXT,
  created_at         TIMESTAMP DEFAULT NOW()
);

```

Example Skill Records:

```

{
  "skill_name": "fetch_source_a_data",
  "tool_id": "<postgres_query_uuid>",
  "instruction": "You are a data fetcher. Use the provided tool to execute the qu",
  "input_schema": {
    "required": ["connection_string", "query"],
    "properties": {
      "connection_string": { "type": "string" },
      "query": { "type": "string" },
      "params": { "type": "array", "default": [] }
    }
  },
  "output_schema": {
    "type": "array",

```

```

        "items": { "type": "object" }
    }
}

{
    "skill_name": "compare_two_sources",
    "tool_id": null,
    "instruction": "You are a comparison agent. Given two datasets (source_a and so
    "input_schema": {
        "required": ["source_a", "source_b", "key_field"],
        "properties": {
            "source_a": { "type": "array" },
            "source_b": { "type": "array" },
            "key_field": { "type": "string" }
        }
    }
}

{
    "skill_name": "verdict_summary",
    "tool_id": null,
    "instruction": "You are a verdict agent. Summarize the comparison results into
    "input_schema": {
        "required": ["comparison_result"],
        "properties": {
            "comparison_result": { "type": "object" }
        }
    }
}

```

3. Personas Registry

A persona is a **role definition**. It groups a set of core skills that always apply to this role, regardless of which use case uses it.

```

CREATE TABLE personas (
    persona_id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    persona_name  VARCHAR(100) UNIQUE NOT NULL,    -- e.g. "data_fetcher", "compar
    description   TEXT,
    core_skills   UUID[] NOT NULL,                -- Array of skill_ids always inc
    system_prompt TEXT,                          -- Base system prompt for this p
    created_at    TIMESTAMP DEFAULT NOW()
);

```

Example Persona Records:

```
{
  "persona_name": "data_fetcher",
  "description": "Fetches data from a specified source using a configured tool.",
  "core_skills": ["<fetch_source_skill_uuid>"],
  "system_prompt": "You are a precise data retrieval agent. Your only job is to f
}

{
  "persona_name": "comparator",
  "description": "Compares two data sources and identifies differences.",
  "core_skills": ["<compare_two_sources_skill_uuid>"],
  "system_prompt": "You are a meticulous data comparison agent. Be thorough and f
}

{
  "persona_name": "verdict_agent",
  "description": "Summarizes comparison results and delivers a final verdict.",
  "core_skills": ["<verdict_summary_skill_uuid>"],
  "system_prompt": "You are an impartial QA verdict agent. Summarize findings cle
}
```

Use Case Configuration

A use case wires together personas, assigns runtime parameter values to their skills, and defines the execution flow.

```
CREATE TABLE use_cases (
  use_case_id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  use_case_name  VARCHAR(100) UNIQUE NOT NULL,
  description    TEXT,
  flow           JSONB NOT NULL,      -- Ordered list of agent steps
  created_at     TIMESTAMP DEFAULT NOW()
);
```

Example Use Case: “Compare Orders Between DB and API”

```
{
  "use_case_name": "compare_orders_db_vs_api",
```



```
"description": "Fetch orders from Postgres and REST API, compare them, then pro
"flow": [
  {
    "step": 1,
    "agent_name": "fetcher_db",
    "persona": "data_fetcher",
    "skill": "fetch_source_a_data",
    "runtime_params": {
      "connection_string": "postgresql://user:pass@host:5432/orders_db",
      "query": "SELECT order_id, amount, status FROM orders WHERE date = $1",
      "params": ["{{ input.date }}"]
    },
    "output_key": "source_a"
  },
  {
    "step": 2,
    "agent_name": "fetcher_api",
    "persona": "data_fetcher",
    "skill": "fetch_source_a_data",
    "runtime_params": {
      "url": "https://api.example.com/orders?date={{ input.date }}",
      "headers": { "Authorization": "Bearer {{ secrets.api_token }}" }
    },
    "output_key": "source_b"
  },
  {
    "step": 3,
    "agent_name": "comparator_agent",
    "persona": "comparator",
    "skill": "compare_two_sources",
    "runtime_params": {
      "source_a": "{{ outputs.source_a }}",
      "source_b": "{{ outputs.source_b }}",
      "key_field": "order_id"
    },
    "output_key": "comparison_result"
  },
  {
    "step": 4,
    "agent_name": "verdict",
    "persona": "verdict_agent",
    "skill": "verdict_summary",
    "runtime_params": {
      "comparison_result": "{{ outputs.comparison_result }}"
    },
    "output_key": "final_verdict"
  }
]
```

```
]
}
```

Orchestrator Design

The orchestrator is the runtime engine. It reads the use case config, loads everything from PostgreSQL, binds parameters, and executes the flow through Google ADK agents.

```
# orchestrator.py - early reference implementation (Google ADK + PostgreSQL)
# NOTE: See three-level orchestration sections for the current architecture.
# This shows the core concept; the actual implementation splits this into
# PersonaOrchestrator, UseCaseOrchestrator, and AgentOrchestrator.

import asyncio
from google.adk.agents import LlmAgent
from google.adk.tools import FunctionTool
from db import get_use_case, get_persona, get_skill, get_tool
from template_engine import resolve_params  # handles {{ }} interpolation

class Orchestrator:

    async def run(self, use_case_name: str, input_data: dict) -> dict:
        use_case = await get_use_case(use_case_name)
        outputs = {}

        for step in sorted(use_case["flow"], key=lambda x: x["step"]):
            result = await self._execute_step(step, input_data, outputs)
            outputs[step["output_key"]] = result

        return outputs

    async def _execute_step(self, step: dict, input_data: dict, outputs: dict) ->
        # 1. Load persona + skill from registry
        persona = await get_persona(step["persona"])
        skill = await get_skill(step["skill"])

        # 2. Resolve runtime parameters (interpolate {{ }} placeholders)
        context = {"input": input_data, "outputs": outputs}
        resolved_params = resolve_params(step["runtime_params"], context)

        # 3. Load and bind tool if skill requires one
        adk_tools = []
        if skill["tool_id"]:
            tool_def = await get_tool(skill["tool_id"])
```

```

    adk_tools.append(build_adk_tool(tool_def, resolved_params))

# 4. Build ADK agent with persona system prompt + skill instruction
agent = LlmAgent(
    name=step["agent_name"],
    model="gemini-2.0-flash",
    instruction=f"{persona['system_prompt']}\n\n{skill['instruction']}",
    tools=adk_tools,
)

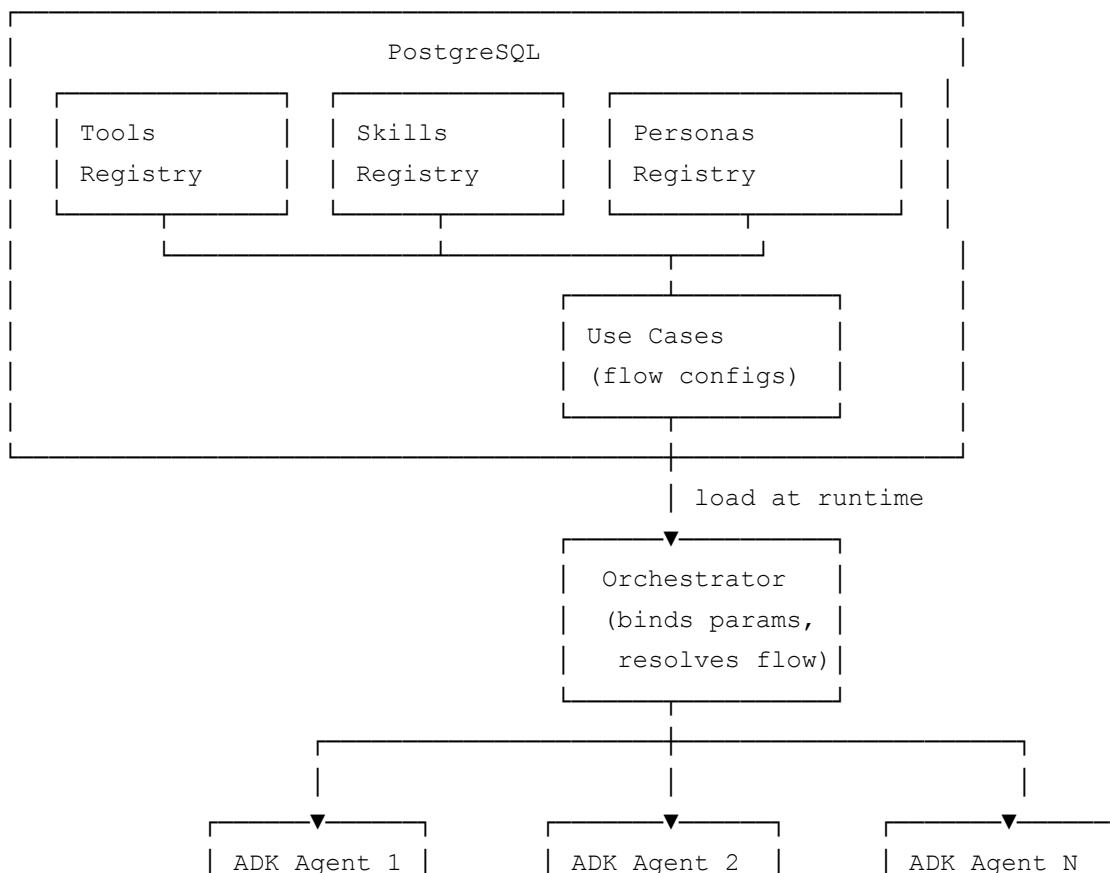
# 5. Run the agent
response = await agent.run(
    user_message=build_user_message(skill, resolved_params)
)

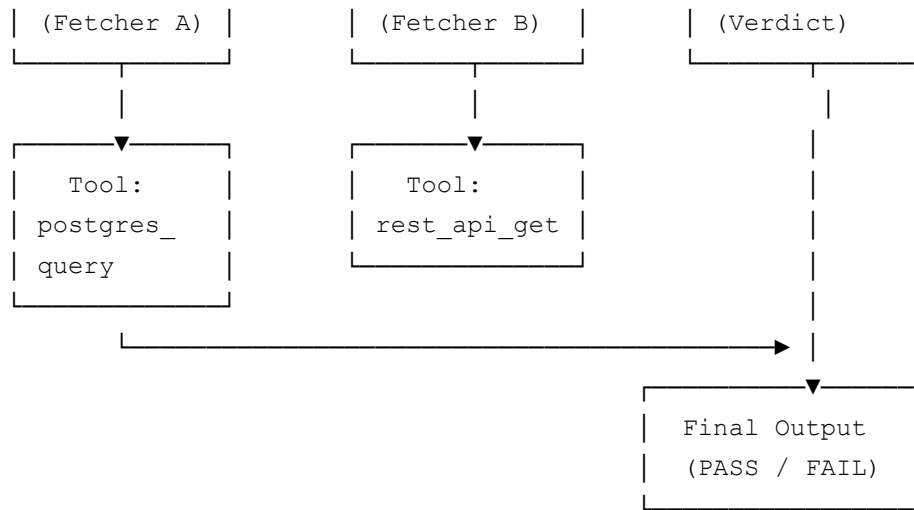
return parse_response(response)

```

System Architecture Diagram

Note: This diagram now reflects the three-level orchestration model. The single “Orchestrator” box has been replaced by three explicit levels: Persona Orchestrator (L1), Use Case Orchestrator (L2), and Agent Orchestrator (L3).





Scaling to Many Use Cases

Because all logic lives in the registries, adding a new use case requires **zero new code**:

1. Register any new tools needed (if not already in registry)
2. Register any new skills (reusing existing tools)
3. Create a new use case record with its flow config
4. The orchestrator handles the rest at runtime

Scenario	What Changes	What Stays the Same
New data source	Add 1 tool record	All skills, personas, orchestrator
New comparison type	Add 1 skill record	All tools, personas, orchestrator
New use case	Add 1 use case record	Everything else
New agent role	Add 1 persona record	All tools, skills, orchestrator

Key Design Principles

1. Skills are templates, not instances. Skills never store actual connection strings, API keys, or specific queries. Those live in the use case's `runtime_params` and are resolved at execution time.

- 2. Personas define roles, not tasks.** A `data_fetcher` persona knows it fetches data — it doesn't know where from. That's determined by the skill + runtime params bound to it per use case.
- 3. The orchestrator is stateless.** It reads config, executes, returns output. No hard-coded use case logic lives in the orchestrator.
- 4. Use `{{ }}` templating for dynamic values.** Runtime parameters support template interpolation so outputs from step N can be passed as inputs to step N+1 without custom wiring code.
- 5. Secrets are separate.** API keys, passwords, and tokens should be resolved via a secrets manager (e.g., Google Secret Manager) using a `{{ secrets.key_name }}` syntax, never stored in the use case config.
-

Summary

Layer	Storage	Purpose
Tools	PostgreSQL <code>tools</code> table	Reusable integrations
Skills	PostgreSQL <code>skills</code> table	Tool + parameter templates
Personas	PostgreSQL <code>personas</code> table	Agent role definitions
Use Cases	PostgreSQL <code>use_cases</code> table	Wiring + runtime values
Use Case Orchestrator	Python service (Google ADK)	Level 2: sequences agents within a use case
Agent Orchestrator	Python service (Google ADK)	Level 3: binds skill + tool + params, instantiates ADK agent
Persona Orchestrator	Python service (Google ADK)	Level 1: chains use cases based on persona definition

Three-Level Orchestration

The Problem It Solves

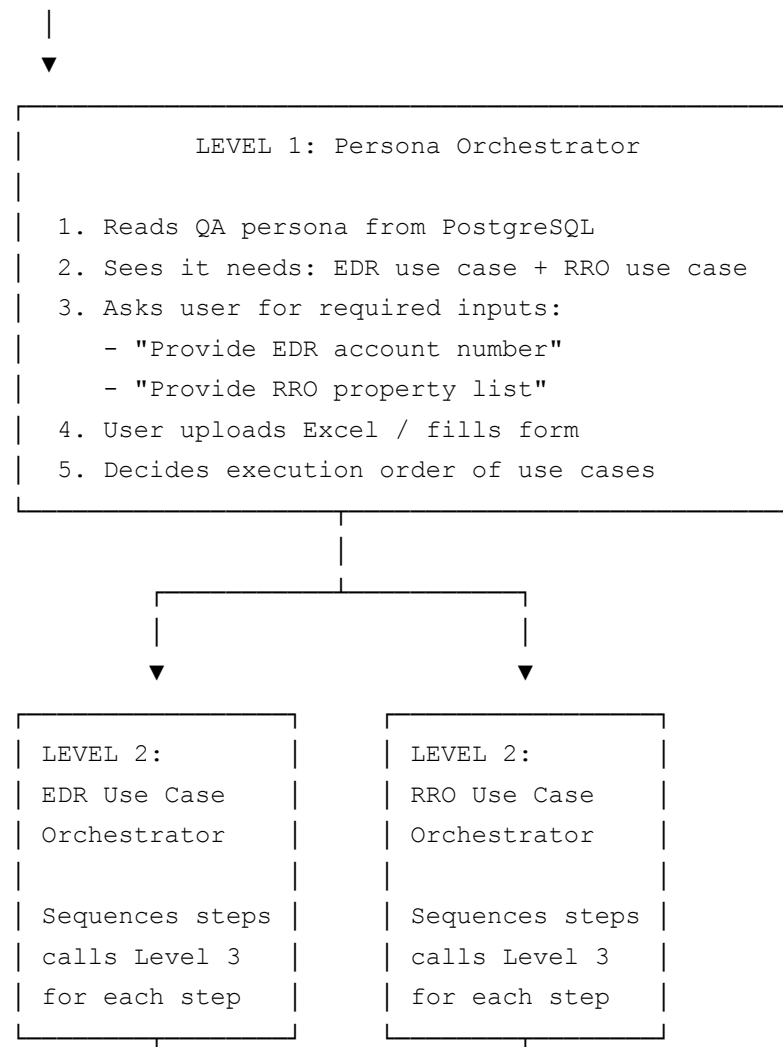
Instead of a user selecting individual use cases (e.g., "run EDR QA" then "run RRO QA"), the user selects a **role** (e.g., "I am QA") and the system figures out what to do. The persona becomes the entry point.

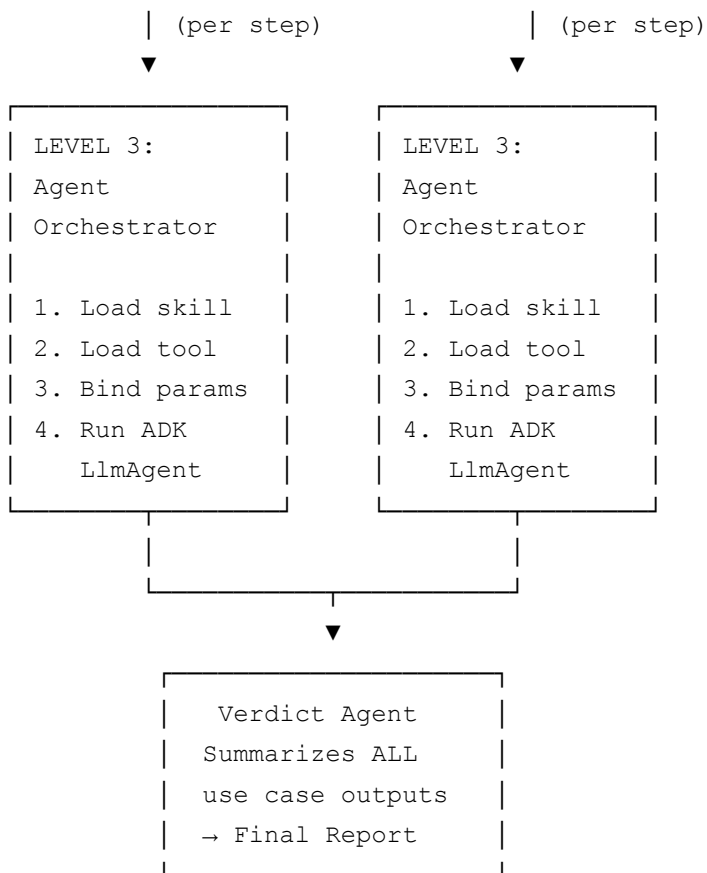
The orchestration has **three distinct levels**, each with a clear responsibility:

- **Level 1 — Persona Orchestrator:** decides which use cases to run and passes mapped inputs
- **Level 2 — Use Case Orchestrator:** sequences agents within a use case, passes outputs between steps
- **Level 3 — Agent Orchestrator:** binds skill + tool + resolved params, instantiates and runs one Google ADK agent

How It Works: End-to-End Flow

User: "I am QA"





PostgreSQL: Persona Definition with Use Cases

```

CREATE TABLE personas (
  persona_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  persona_name    VARCHAR(100) UNIQUE NOT NULL,
  description     TEXT,
  system_prompt   TEXT,
  use_cases       JSONB NOT NULL,    -- ordered list of use cases + input mappin
  input_schema    JSONB NOT NULL,    -- what the user must provide to run this p
  created_at      TIMESTAMP DEFAULT NOW()
);

```

Example QA Persona Record:

```

{
  "persona_name": "qa_agent",
  "description": "QA persona that runs EDR and RRO validation end-to-end.",
  "system_prompt": "You are a QA orchestrator. Run all assigned use cases and pro
  \"input_schema\": {
    \"required\": [\"edr_account_number\", \"rro_property_ids\"],
    \"properties\": {
      \"edr_account_number\": { \"type\": \"string\", \"label\": \"EDR Account Number\" },

```

```

        "rro_property_ids": { "type": "array", "label": "RRO Property IDs (uploa
    }
},
"use_cases": [
    {
        "order": 1,
        "use_case_name": "edr_qa",
        "input_mapping": {
            "account_number": "{{ persona_input.edr_account_number }}"
        }
    },
    {
        "order": 2,
        "use_case_name": "rro_qa",
        "input_mapping": {
            "property_ids": "{{ persona_input.rro_property_ids }}"
        }
    }
]
}

```

Persona Orchestrator (Python + Google ADK)

```

# persona_orchestrator.py

class PersonaOrchestrator:

    async def run(self, persona_name: str, user_input: dict) -> dict:
        # 1. Load persona from PostgreSQL
        persona = await get_persona(persona_name)

        # 2. Validate user_input against persona's input_schema
        validate(user_input, persona["input_schema"])

        all_outputs = {}

        # 3. Execute each use case in order
        for uc_config in sorted(persona["use_cases"], key=lambda x: x["order"]):

            # 4. Resolve input mapping (bind persona inputs to use case inputs)
            uc_input = resolve_params(uc_config["input_mapping"], {
                "persona_input": user_input,
                "outputs": all_outputs
            })

            # 5. Hand off to use case orchestrator (Level 2)

```



```

        uc_result = await UseCaseOrchestrator().run(
            use_case_name=uc_config["use_case_name"],
            input_data=uc_input
        )

        all_outputs[uc_config["use_case_name"]] = uc_result

# 6. Run verdict agent across all use case outputs
verdict = await run_verdict_agent(all_outputs)

return {
    "persona": persona_name,
    "use_case_outputs": all_outputs,
    "verdict": verdict
}

```

Where Skills Fit In

Skills are invoked at the **Agent Orchestrator level** (Level 3), not at the persona or use case level. The Agent Orchestrator is the only component that directly touches the Skills Registry and Tools Registry at runtime. It loads the skill instruction (L2 progressive disclosure), binds the tool with resolved parameters, and instantiates the Google ADK agent.

Persona Orchestrator (Level 1)

→ decides WHICH use cases to run, maps user inputs to each use case

Use Case Orchestrator (Level 2)

→ sequences agents step by step, passes outputs between steps

→ calls Level 3 once per step

Agent Orchestrator (Level 3)

← NEW explicit layer

→ loads SKILL instruction from Skills Registry (L2 progressive disclosure)

→ loads TOOL definition from Tools Registry

→ binds resolved runtime params (connection_string, query, URL, etc.)

→ instantiates Google ADK LlmAgent(instruction, model, tools)

→ executes agent and returns structured output to Level 2

Three-Level Summary

Level	Name	Who Calls It	Responsibility
Level 1	Persona Orchestrator	User / UI	Receives user input, validates against persona input schema, chains use cases in order

Level 2	Use Case Orchestrator	Persona Orchestrator	Loads use case flow from PostgreSQL, iterates steps, resolves <code>{{ }}</code> templates, calls Level 3 per step
Level 3	Agent Orchestrator	Use Case Orchestrator	Loads skill (L2), loads tool, binds resolved params, instantiates Google ADK LlmAgent, executes and returns output
ADK LlmAgent	Google ADK Runtime	Agent Orchestrator	Executes the actual LLM reasoning + tool call
Skill Registry	PostgreSQL	Agent Orchestrator (L3)	Provides skill instructions and metadata on demand
Tools Registry	PostgreSQL	Agent Orchestrator (L3)	Provides tool definitions for binding
Verdict Agent	ADK Agent (via L3)	Use Case Orchestrator	Summarizes all use case outputs into a final PASS/FAIL report

Progressive Disclosure in Your Architecture

The Agent Skills standard defines three loading levels that minimize context token usage. This architecture applies the same pattern at every layer.

The Three Levels

Level	What Loads	When It Loads	Token Cost
L1 — Metadata	<code>name</code> + <code>description</code> only	At agent startup, always	~100 tokens per skill
L2 — Instructions	Full <code>SKILL.md</code> body	When skill is activated for a task	< 5000 tokens recommended
L3 — Resources	Scripts, schemas, reference files	Only when the skill explicitly needs them	On demand

How This Applies to Your System

L1 — Skill Discovery (Startup)

When the orchestrator boots or a persona is activated, it loads only the `skill_name` and `description` for all skills from PostgreSQL. This lets the persona orchestrator decide which skills are relevant without loading full instructions.

```
# L1: load only name + description for all skills
skills_index = await db.fetch(
    "SELECT skill_name, description FROM skills"
)
# ~100 tokens per skill — cheap to load hundreds
```

L2 — Skill Activation (When Agent Runs)

When an agent is assigned a skill, the full instruction body is loaded. This is the `instruction` field in your skills table — equivalent to the SKILL.md body content.

```
# L2: load full instructions only when skill is needed
skill = await db.fetchrow(
    "SELECT instruction, allowed_tools, compatibility FROM skills WHERE skill_name = %s"
)
# Load once per agent execution
```

L3 — Resource Loading (On Demand)

Input/output schemas, scripts, and reference files are loaded only when the agent explicitly needs them — for example when validating parameters or running a helper script.

```
# L3: load schema only when validating inputs
if needs_validation:
    schema = await db.fetchrow(
        "SELECT input_schema, output_schema FROM skills WHERE skill_name = %s",
        skill_name
    )
```

Updated Skills Table Supporting Progressive Disclosure

```
CREATE TABLE skills (
    skill_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    -- L1: Always loaded at startup
    skill_name        VARCHAR(100) UNIQUE NOT NULL,    -- lowercase-hyphenated
    description        TEXT NOT NULL,                    -- max 1024 chars, WHAT + WHEN
```

```

-- L2: Loaded when skill is activated
instruction      TEXT NOT NULL,          -- SKILL.md body, < 5000 token
allowed_tools    TEXT,                  -- space-separated tool names
compatibility     VARCHAR(500),

-- L3: Loaded on demand
input_schema     JSONB,
output_schema     JSONB,
tool_id          UUID REFERENCES tools(tool_id),

-- Standard metadata
metadata         JSONB DEFAULT '{}',    -- version, author, etc.
license          VARCHAR(200),
created_at       TIMESTAMP DEFAULT NOW()
);

```

Progressive Disclosure Flow in Your Multi-Agent System

```

Persona Orchestrator starts
|
|— L1: Load name + description for ALL skills      (~100 tokens each)
|   → Decides which skills are relevant
|
|— Persona activates Use Case Orchestrator
|   |
|   |— L2: Load full instruction for EACH agent's skill
|   |   → Builds ADK agent with full context
|   |
|   |— L3: Load input_schema when validating params
|   |   Load output_schema when parsing results
|   |   Load scripts only if skill needs them
|
|— Verdict Agent summarizes all outputs

```

Benefits for Your Architecture

By applying progressive disclosure you get three concrete benefits. First, you can register hundreds of skills without slowing down agent startup, since only names and descriptions load at boot. Second, each agent only pays the token cost for its own skill instructions, not all skills in the registry. Third, large reference documents and schemas stay on disk or in PostgreSQL and are fetched only when actually needed, keeping your context window lean throughout the entire workflow.